

A versatile door opening system with mobile manipulator through adaptive position-force control and reinforcement learning

Gyuree Kang^a, Hyunki Seong^a, Daegyu Lee^b, David Hyunchul Shim^{a,*}

^a School of Electrical Engineering, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea

^b Digital Convergence Research Laboratory, Electronics and Telecommunications Research Institute, Daejeon, Republic of Korea

ARTICLE INFO

Keywords:

Mobile manipulator
Door opening robot
Real-time autonomous system
Indoor robotics
Deep reinforcement learning

ABSTRACT

The ability of robots to navigate through doors is crucial for their effective operation in indoor environments. Consequently, extensive research has been conducted to develop robots capable of opening specific doors. However, the diverse combinations of door handles and opening directions necessitate a more versatile door opening system for robots to successfully operate in real-world environments. In this paper, we propose a mobile manipulator system that can autonomously open various doors without prior knowledge. By using convolutional neural networks, point cloud extraction techniques, and external force measurements during exploratory motion, we obtained information regarding handle types, poses, and door characteristics. Through two different approaches, adaptive position-force control and deep reinforcement learning, we successfully opened doors without precise trajectory or excessive external force. The adaptive position-force control method involves moving the end-effector in the direction of the door opening while responding compliantly to external forces, ensuring safety and manipulator workspace. Meanwhile, the deep reinforcement learning policy minimizes applied forces and eliminates unnecessary movements, enabling stable operation across doors with different poses and widths. The RL-based approach outperforms the adaptive position-force control method in terms of compensating for external forces, ensuring smooth motion, and achieving efficient speed. It reduces the maximum force required by 3.27 times and improves motion smoothness by 1.82 times. However, the non-learning-based adaptive position-force control method demonstrates more versatility in opening a wider range of doors, encompassing revolute doors with four distinct opening directions and varying widths.

1. Introduction

With the increasing use of robots in various aspects of daily life, their working environments are expanding beyond well-defined spaces to include unstructured settings such as office buildings and streets. Therefore, their ability to interact effectively with their surroundings has become essential. While autonomous mobile robots have found applications in human-centered fields such as delivery services [1,2], their accessibility to human-oriented facilities, particularly doors, remains a challenge. Hence, there have been numerous studies on door opening using mobile manipulators [3–5]. However, many of these studies have focused on investigating approaches for opening a certain type of door or examining isolated parts of the door opening process, such as handle detection and pose estimation, handle unlatching, or door opening. Additionally, validation in real-world environments is often lacking, as most evaluations have been conducted solely in simulations. Therefore, in this study, we propose a system utilizing a mobile manipulator to perform the complete process of door opening and passage. Our

approach aims to address these limitations by considering various types of door handles, opening directions, and door widths and evaluating them in the real world.

The challenge of opening doors arises from the kinematically constrained situation of the robot while holding the door. The lack of compliance in the robot's hardware can lead to dangerous levels of force being exerted on both the door and the robot even with slight deviations in their movements. To address this challenge, the robot must understand the kinematic constraints of the target door, including its rotation axis, opening direction, and the kinematics of the handles. [6] utilized online estimation of the door radius to enable a mobile manipulator to perform rotational door opening movements. They employed a hybrid dynamic system that incorporated compliant motion, door model predictive control, and ideal manipulator configuration control. [7] adopted multiple working modes for the joints to prevent excessive internal forces during the target door opening. They switched the joints parallel to the hinge to passive mode and employed a door

* Corresponding author.

E-mail addresses: fingb20@kaist.ac.kr (G. Kang), hynkis@kaist.ac.kr (H. Seong), lee.dk@etri.re.kr (D. Lee), hcshim@kaist.ac.kr (D.H. Shim).

<https://doi.org/10.1016/j.robot.2024.104760>

Received 28 June 2023; Received in revised form 12 January 2024; Accepted 17 July 2024

Available online 19 July 2024

0921-8890/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



Fig. 1. Mobile manipulator opening various doors.

parameter estimation method based on recorded motion to generate the end-effector trajectory. While these studies estimated the target door's radius online to generate the robot trajectory, allowing the system to be applicable to different door types, they only tested the system on pulling-type doors and did not provide a solution to other parts of the door opening process. [8] focused on optimizing the energy consumption during the door opening motion. However, their manipulator trajectory was generated based on the measured door radius, making the system dependent on the accuracy of sensor measurements. Meanwhile, [9] adopted impedance control and convolutional neural networks (CNN) for handle detection in opening a door with a mobile manipulator. They also considered the robot's traversability while opening the door and were able to unlatch handles with different shapes. However, their system was only deployed on a single type of handle and a pushing door. [10] also used CNN to detect the target handle and door, while estimating the pose with a random sample consensus (RANSAC) algorithm. Then, they estimated door kinematics for prismatic and revolute type doors based on the Bayesian approach and utilized human demonstration as prior knowledge. This approach allowed the system to unlatch different types of handles but did not consider passing through the door.

However, designing planning and control systems for highly complex systems, such as mobile manipulators, is a challenge for conventional methods, as they are sensitive to parameters and require significant engineering effort [11]. To address this issue, reinforcement learning (RL) has emerged as a promising approach for complex robot control [12–14], including mobile manipulators [15–18]. As it can be less sensitive to hardware calibration [19], generate robot actions

directly from low-level sensor observations, and learn the policy by itself without much prior knowledge and fine-tuning [20], it is suitable for tasks such as door opening. [21] developed a learning-based door opening robot, where they adopted asynchronous NAF (Normalized Advantage Function) to train the network with experience collected by multiple robots in the real-world. This enabled faster learning of the robot for complex manipulation in the real-world application without prior demonstrations. Meanwhile, [22] sped up the training of a manipulator for door opening by using physical constraints of the task as a guideline for robot motion planning. The intelligent control algorithm used in searching RL policy parameters, and the training result was tested both in simulation and in the real world. Both studies successfully implemented the agent in the real world but used a fixed manipulator to open doors. [23] took the domain randomization problem for the door opening task a step further by introducing a simulation environment with different door parameters and robot properties. They also tested the simulation by training the agent with the *Proximal Policy Optimization* (PPO), and *Soft Actor-Critic* (SAC) algorithms, which showed a success rate of up to 0.95 when tested in a randomized environment. However, they focused only on the door opening without integration into the door-passing task. [24] showed door opening with a mobile manipulator, where RL was used for generating the door opening motion. They used an improved PPO algorithm for training but only focused on a single type of door and handle without utilizing the mobile part of the robot to pass through or approach the door. Other than RL, [25] used deep predictive learning (DPL) to open and pass through the door with the mobile manipulator. The task was divided into three parts: approaching, opening,

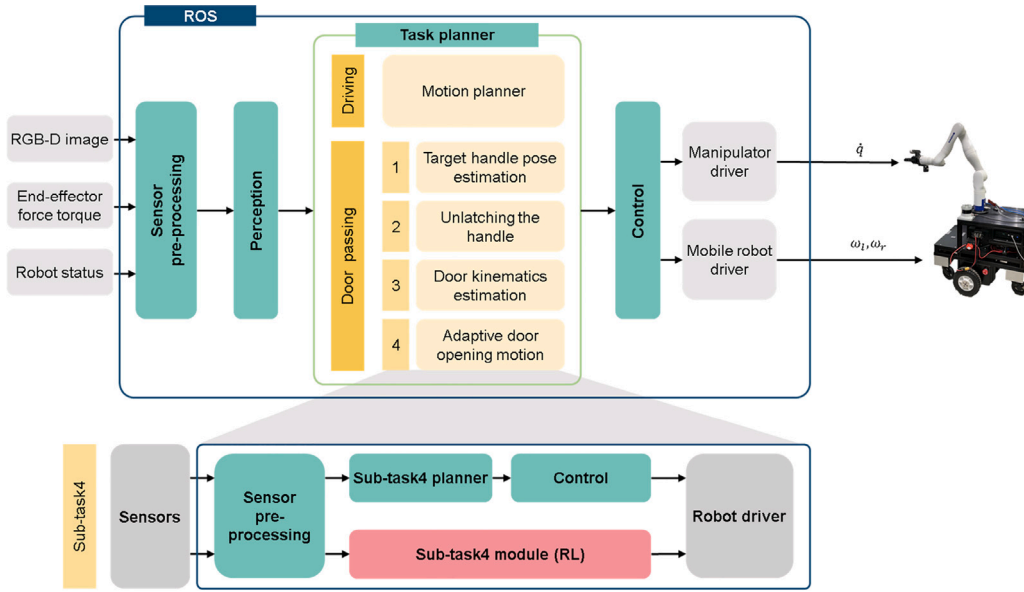


Fig. 2. The mobile manipulator door opening system architecture. The controller in subtask 4 (ST4) for adaptive door opening motion was designed using two different approaches: adaptive position-force controller, and deep reinforcement algorithm.

and passing, while each part was executed with the proposed motion generation module. The modules generated predictive images and robot motions for the near future, and the one with the lowest standardization error was chosen for the final execution. However, their study did not demonstrate generalization ability in terms of door type, showing only a single door opening case.

In this paper, we propose a versatile door opening system capable of opening doors with various handle types and opening directions (Fig. 1). The system consists of a perception module that estimates the class and pose of the target handle from an RGBD image using a segmentation algorithm. Additionally, the robot estimates door kinematics through exploratory motion and force measurement with a force/torque sensor at the end-effector. During the door opening and passing phases, we implement and compare non-learning adaptive position-force control and RL-based techniques, as they are two of the most extensively employed methods for adaptive door opening. The adaptive position-force control method, known for its ease of adjustability and adaptability to different door types, proved to be less sensitive to environmental variations. Leveraging information from preceding modules, it successfully opened doors with different handle types, opening directions, and widths. In contrast, the RL method demonstrated its effectiveness in achieving diverse objectives, considering intricate robot actions, and utilizing various sensor inputs by exhibiting superior performance in terms of compliance, stability, and speed when dealing with pushing doors of varying widths.

The contributions of this study are as follows.

- Proposing a versatile full-sequence door opening system framework for mobile manipulators, applicable to doors with various handles and kinematic constraints without prior knowledge.
- Designing both adaptive position-force-based system and RL-based hybrid system for adaptive door opening.
- Implementing both systems and extensively evaluating them in real-world scenarios.

The rest of the paper is organized as follows: In Section 2, we provide a brief overview of the four subtask modules, followed by a detailed description of each module in Sections 2.1–2.5. In particular, Sections 2.4 and 2.5 explain the two approaches for adaptive door opening and passing motion. We describe the experiment settings in Section 3, and present the simulation and real-world test results in Section 4. Finally, we provide our conclusions in Section 5.

2. Methods

To pass through a door using a mobile manipulator, we divide the task into four subtasks, which are solved with corresponding modules as shown in Fig. 2. The first subtask is estimating the pose and the type of the target handle (ST1) and approaching it until it is within reach of the manipulator. The second subtask is unlatching the handle (ST2) based on the type defined from ST1. The third subtask is identifying the opening direction of the door (ST3). The fourth subtask is opening the door and passing through it (ST4) according to the opening direction identified in ST3, ensuring no collision. The door-opening scenario initiates when the robot encounters a target door and approaches it in driving mode. It finishes when the door is fully opened, prompting the robot to switch back to autonomous driving mode.

2.1. Handle pose estimation

Since door handles are small objects usually ranging from 20 cm to 10 cm in width, we combine object detection results from images with depth images to generate point clouds of the target rather than detecting the target directly from the point cloud [26] as in Fig. 3. A real-time image segmentation algorithm [27] is utilized to detect and classify the handles into three types: lever type (type 1), door knob (type 2), and bar handle (type 3). To generate the point cloud of the target handle from the depth image, the segmented image of the handle is used as a mask M_h to filter out the background from the depth image. The filtered depth image is then converted into a point cloud (${}^{cam}P_h = p_1^d, p_2^d, \dots, p_j^d$) of the target handle through depth image projection.

The pose of the handle is estimated using ${}^{cam}P_h$. The center of ${}^{cam}P_h$ is assumed to be the position of the target, whereas the vertical axis from the RANSAC plane segmentation algorithm [28] is used to estimate the orientation. The estimated pose ${}^{cam}X_{trg}$ of the handle is then transformed to the base frame ${}^{mb}X_{trg}$ using T_{cam}^{mb} , the transformation matrix from the camera frame to the base frame. Once the pose of the handle has been estimated, the robot approaches ${}^{mb}X_{trg}$ until it is within reach (ρ) of the manipulator, while the manipulator aligns its pose relative to ${}^{mb}X_{trg}$ in preparation for the handle unlatching motion. This method of pose estimation is compatible with handles with various shapes, as it does not require CAD or any other prior knowledge of the target object.

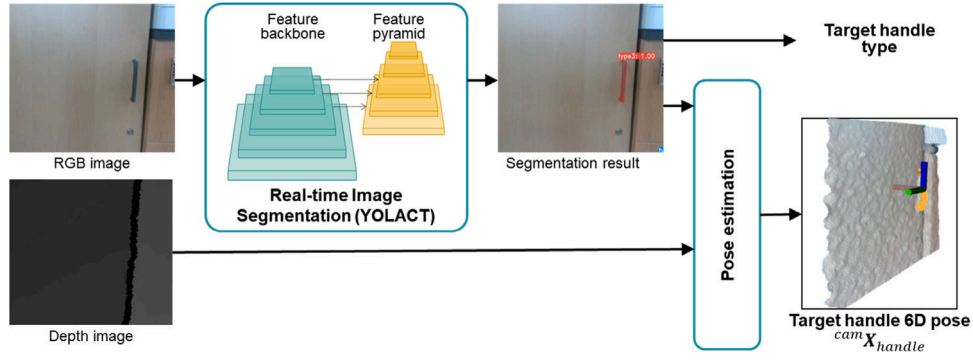


Fig. 3. Target handle type and pose estimation algorithm. The image segmentation result serves as a mask to filter out the depth image, which is subsequently projected to the point cloud of the target handle. The point cloud is utilized to estimate the pose of the target handle.

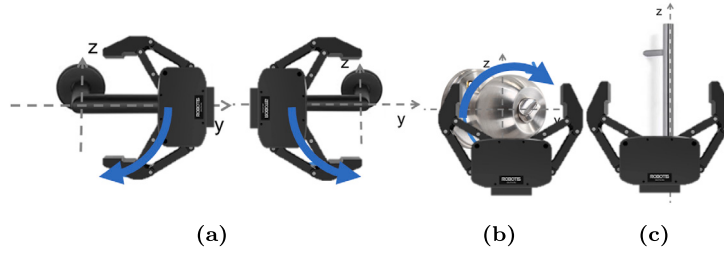


Fig. 4. Predefined grasping poses according to handle types. (a) type1: levers, (b) type2: doorknobs, and (c) type3: pulling handles.

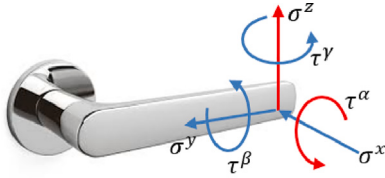


Fig. 5. Unlatching handle motion with adaptive position-force control. The red axes represent the manipulation axes, while the blue axes represent the passive axes. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

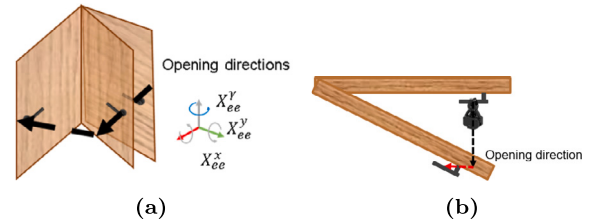


Fig. 6. Estimating the opening direction of the door based on exploratory motion and force measurements. (a) motion axis (x , y , and z) for exploratory motion, (b) confirmed opening direction.

2.2. Unlatch handles

To unlatch the handle, the robot positions the end-effector to a grasping pose according to the target handle's type as shown in Fig. 4. To avoid the target getting out of the camera's view and being unable to identify the object when it gets too close, the end-effector slowly reaches out until the handle is detected by the wrist force/torque sensor. Once the manipulator contacts with the handle, the gripper grasps the target.

After grasping the handle, the robot unlatches it using end-effector adaptive position-force control described in Section 2.4. The robot exerts force and generates a target position in the manipulation directions of the handle as in Fig. 5. The force limits of the manipulation axes for the adaptive position-force control are set high to allow the robot to apply force to unlatch the handle. Conversely, the limits of the passive (non-manipulation) axes are set low to generate compensating motion for the external force to prevent excessive loading. The robot continues the unlatching motion until the force/torque of the end-effector reaches its thresholds (Ω , Λ) on the manipulation axis.

2.3. Door kinematics estimation

The door kinematics comprise the normal opening direction (ψ^x) and the horizontal direction (ψ^y), as shown in Fig. 6. Initially, the robot

generates the exploratory motion in x -axis to estimate ψ^x by pushing the door. If the force measurement in the x -axis does not exceed Ω^x , the door is considered a push-open door ($\psi^x = 1$). Otherwise, the robot pulls the door, and if the same condition ($|\sigma^x| < \Omega^x$) is met, the door is defined as a pull-open door ($\psi^x = -1$).

After defining ψ^x , the robot opens the door while generating compliant movement to external adaptive position-force control in the passive axis of the door. If the end-effector moves farther than the threshold in the x -axis (δ^x) and relatively left to the previous pose, the horizontal opening direction of the door ψ^y is defined as 1; otherwise, it is defined as -1 . The door is considered a clockwise (CW) door if $\psi^x\psi^y$ is 1; otherwise, it is considered a counter-clockwise (CCW) door.

2.4. Opening and navigating through doors: Adaptive position-force-based method

As depicted in Fig. 7, the adaptive position-force controller comprises four modules: a position controller, a workspace checker, a force/torque feedback controller, and an inverse kinematic solver (IKS). Initially, the robot generates the end-effector goal pose towards the opening direction (ψ^x, ψ^y) of the door. The position controller receives the end-effector's target position ${}^{mb}X_{irg}$ and computes the desired end-effector velocity ${}^{mb}\dot{X}_{ee,pos}$ using the PID control method with the current end-effector pose as the feedback.

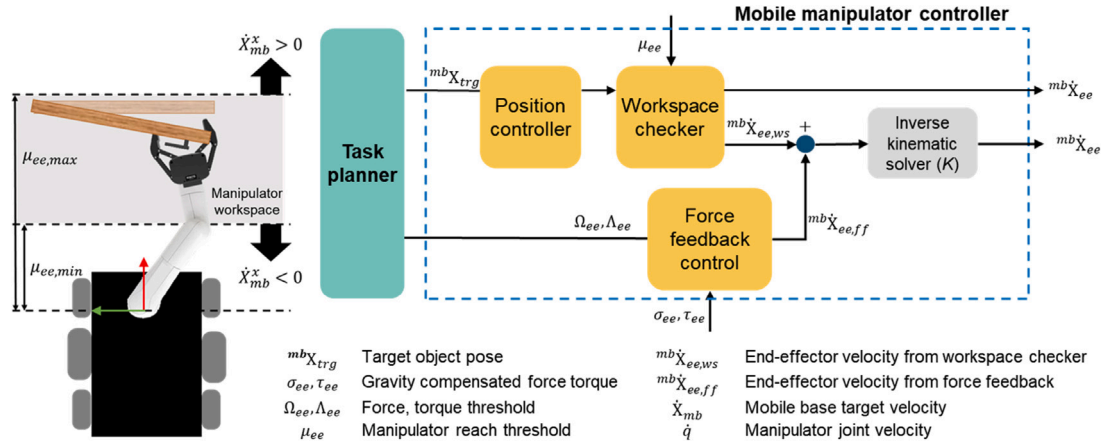


Fig. 7. The adaptive position-force control system of the mobile manipulator consists of two controllers. The position controller adjusts the end-effector's pose according to the pose of the target object, which is verified by the workspace checker to ensure that it is within the manipulator's reach and the joint position limits. The adaptive position-force controller utilizes force/torque data from the sensor to adaptively control the end-effector's motion in response to external forces.

Then the workspace checker detects potential collisions between the door and checks if the goal point is within the manipulator's workspace. If the target point is farther than the workspace of the manipulator ($[\mu_{ee,min}, \mu_{ee,max}]$), the mobile robot moves towards the goal point. However, if the manipulator and the mobile robot are close enough to predict a potential collision ($\mu_{ee,min}$), the mobile robot moves backward to secure space between them while keeping the end-effector velocity constant.

$${}^{mb}\dot{X}_{ee,ws} = {}^{mb}\dot{X}_{ee,pos} - \dot{X}_{mb} \quad (1)$$

The force controller adjusts the target of the end-effector when the gravity-compensated force or torque at the end-effector (σ_{ee}, τ_{ee}) exceeds its limits ($\Omega_{ee}, \Lambda_{ee}$). Here, the limits for the manipulation axes (x, y, γ) are set high, allowing the robot to apply force to the door. In contrast, the limits for the passive axes (z, α, β) are set low, enabling the end-effector pose to adjust to external forces and reducing unnecessary power consumption. The force controller generates a counter velocity ${}^{mb}\dot{X}_{ee,ff}$ to compensate for the external force as in Eq. (2).

$${}^{mb}\dot{X}_{ee,ff} = K_{p,cmp}\sigma_{err} + K_{i,cmp}\int \sigma_{err}dt \quad (2)$$

Here, σ_{err} is the error between σ_{ee} and the force/torque limit ($\Omega_{ee}, \Lambda_{ee}$). Next, the compliant velocity is added to the target velocity using Eq. (3), resulting in the final end-effector velocity ${}^{mb}\dot{X}_{ee}$ and the mobile velocity $\dot{X}_{mb}$.

$${}^{mb}\dot{X}_{ee} = {}^{mb}\dot{X}_{ee,ws} + {}^{mb}\dot{X}_{ee,ff} \quad (3)$$

Finally, the joint velocity ($\dot{q} = \mathcal{K}({}^{mb}\dot{X}_{ee})$) and wheels' angular velocity (ω_l, ω_r) are generated from the robot driver when ${}^{mb}\dot{X}_{ee}$ and $\dot{X}_{mb}$ are given as input. Here, IKS function $\mathcal{K}$ finds the control values for the corresponding joint velocity $\dot{q}$ given a desired end-effector velocity ${}^{mb}\dot{X}_{ee}$ using Jacobian matrix [29].

The robot considers the door fully opened when the y-distance of the handle from the robot center exceeds half of the robot's width, ensuring that the door no longer obstructs the robot. Subsequently, the mobile part of the robot moves forward while the manipulator holds the handle to prevent the door from closing. The gripper then releases the handle and moves to the other side of the door where the base is located. Finally, as the robot moves forward to pass through the door, the manipulator pose is initialized (q_{init}), and the task mode changes from door-passing to autonomous driving mode.

2.5. Opening and navigating through doors: Deep reinforcement learning-based method

2.5.1. States and actions

The robot's state s_t includes the environment and the robot's status. For the environment state vector, we use distance from the LiDAR point cloud which is the representation of the surrounding obstacles. To reduce the computational complexity of the input, we use only the closest points located within the front half of the robot in 9-degree intervals ($S = \{s_1, s_2, \dots, s_{20}\}$).

For the manipulator's state, we use the measured joints' position ($\hat{q} = \{\hat{q}_1, \hat{q}_2, \dots, \hat{q}_7\}$), velocity ($\hat{\dot{q}} = \{\hat{\dot{q}}_1, \hat{\dot{q}}_2, \dots, \hat{\dot{q}}_7\}$), and effort ($\hat{e} = \{\hat{e}_1, \hat{e}_2, \dots, \hat{e}_7\}$). To jointly control the mobile base and the manipulator, the 6D pose of the end-effector relative to the mobile base is included ($\hat{X}_{ee}^{mb} \in \mathbb{R}^6$). Finally, the mobile robot state is represented by the position and orientation relative to the initial pose of the door ($\hat{X}_{mb} \in \mathbb{R}^3$) as well as its velocity ($\hat{\dot{X}}_{mb} \in \mathbb{R}^3$).

$$s_t = \{\hat{q}, \hat{\dot{q}}, \hat{e}, \hat{X}_{ee}^{mb}, S, \hat{X}_{mb}, \hat{\dot{X}}_{mb}\}_t \in \mathbb{R}^{53} \quad (4)$$

The action space includes the joint velocity of the manipulator and the longitudinal and angular velocity of the mobile robot. Since the manipulator has seven degrees of freedom and the mobile robot has two, there are a total of nine actions as follows:

$$a_t = \{\dot{q}, \dot{X}_{mb}\}_t \in \mathbb{R}^9 \quad (5)$$

2.5.2. Reward design

Two main terms are used in reward (r_t) design to successfully open doors with stable motion: the task rewards $r_{mss,t}$, and the control penalties $r_{ctl,t}$.

$$r_t = r_{mss,t} + r_{ctl,t} \quad (6)$$

For the $r_{mss,t}$, four task-related rewards are added with weights balancing: door open reward $r_{opn,t}$, door passing reward $r_{pss,t}$, obstacle distance reward $r_{obs,t}$, and collision penalty $r_{col,t}$.

$$r_{mss,t} = w_{opn}r_{opn,t} + w_{pss}r_{pss,t} + w_{obs}r_{obs,t} + w_{col}r_{col,t} \quad (7)$$

$r_{opn,t}$, illustrated in Fig. 8(a), is determined based on the extent to which the agent has opened the door in a single step. It is proportional to the difference in the door angle between steps, which can be obtained from the ground truth in the simulation.

$$r_{opn,t} = \theta_t - \theta_{t-1} \quad (8)$$

How much the robot succeeded in passing the door is reflected in $r_{pss,t}$ (Fig. 8(b)), which includes the distance and angle of the robot with

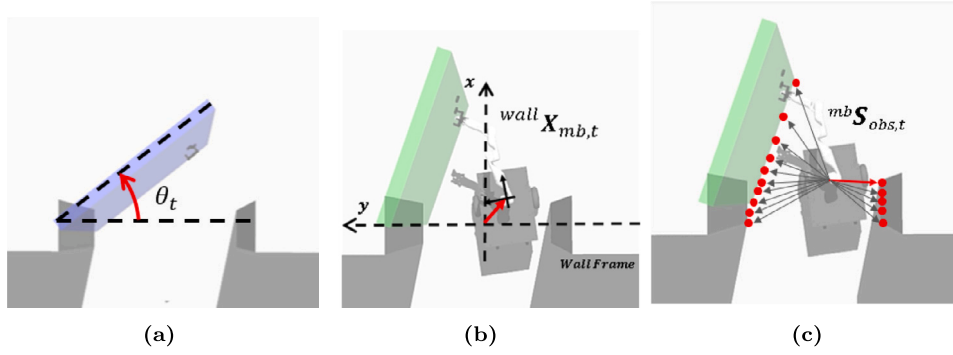


Fig. 8. Task reward definition. (a) door opening reward, (b) door passing reward, and (c) obstacle distance reward.

respect to the wall. $r_{obs,t}$ in Fig. 8(c) represents the distance between the closest obstacle and the robot, which can be obtained from $\mathcal{S}$ in the state representation s_t .

$$r_{obs,t} = -\min(S_{i,t}) \quad (9)$$

Finally, a collision penalty ($r_{col,t}$) of -1 was added whenever a self-collision or a collision between the robot and an obstacle occurs.

The control penalties include joint effort penalty $r_{eff,t}$, joint position penalty $r_{pos,t}$, and robot action scale penalty $r_{act,t}$.

$$r_{ctl,t} = w_{eff}r_{eff,t} + w_{pos}r_{pos,t} + w_{act}r_{act,t} \quad (10)$$

Each penalty is scaled in $[-1, 0]$, and is expressed in polynomial as shown in Eq. (11), so that it can increase rapidly around the limit value. $r_{pos,t}$ and $r_{eff,t}$ include joint position penalty and effort penalty for the seven joints of the manipulator. Meanwhile, $r_{act,t}$ includes both the manipulator and mobile robot actions, aiming to smooth out movement and improve the energy efficiency of the robot by preventing unnecessary actions.

$$r_{eff,t} = -\sum_{j=1}^7 w_{e,j} (e_{j,t})^2 \quad (11)$$

2.5.3. Algorithm and architecture

We use the soft actor-critic (SAC) algorithm [30] with the double Q-learning scheme due to its sample efficiency and stability concerning hyperparameter tuning, which has been proven in the field of various robotic applications, including autonomous driving [31,32] and manipulator control [33,34]. Algorithm 1 summarizes our overall training process with the hyperparameter configuration in Table 1. Following the approach in [35,36], we implement an actor-critic framework that consists of a policy network π_ϕ and two Q-value networks Q_{θ_1} and Q_{θ_2} for the double Q-learning approach. In each episode loop, we first obtain state information s_t from the manipulator and mobile robot. Our policy then infers an action a_t and interacts with the environment to get state transition data $(s_t, a_t, r(s_t, a_t), s_{t+1})$, which is saved in a replay buffer D . During the update phase, the SAC algorithm optimizes each of the Q-value networks by minimizing the Bellman error of the soft Q-function ($J_Q(\theta_i)$). Then the algorithm updates the policy by minimizing the objective function $J_\pi(\phi)$ composed of the expected return and entropy bonus as shown below:

$$J_\pi(\phi) = -\mathbb{E}_{s_t \sim D} [\mathbb{E}_{a_t \sim \pi_\phi} [\min_{i=1,2} Q_{\theta_i}(s_t, a_t) + \alpha H(\pi_\phi)]], \quad (12)$$

where s_t and a_t are sampled to compute the Q-values from the buffer D and policy π_ϕ , respectively, and the minimum of the two Q-values is chosen to mitigate value overestimation. The entropy term $H(\pi_\phi)$, with an automatically tunable coefficient α , acts as a regularizer to maintain proper uncertainty in the policy model. By balancing the clipped Q-value and entropy of the policy, the SAC algorithm achieves a trade-off between maximizing the expected return and exploring near-optimal policies.

Algorithm 1 SAC with the double Q-learning scheme.

Input: ϕ, θ_1, θ_2

Results: Optimized parameters ϕ, θ_1, θ_2

- 1: Initialize parameters ϕ, θ_1, θ_2 and dataset $D \leftarrow \emptyset$
- 2: **for** each episode **do**
- 3: **while** not done **do**
- 4: $a_t \sim \pi_\phi(a_t | s_t)$
- 5: $s_{t+1} \sim p(s_{t+1} | s_t, a_t)$
- 6: $D \leftarrow D \cup \{(s_t, a_t, r(s_t, a_t), s_{t+1})\}$
- 7: **for** each gradient step **do**
- 8: $\theta_i \leftarrow \theta_i - \lambda_Q \nabla_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$
- 9: $\phi \leftarrow \phi - \lambda_\pi \nabla_\phi J_\pi(\phi)$
- 10: $\alpha \leftarrow \alpha - \lambda_\alpha \nabla_\alpha J(\alpha)$

Table 1

Hyper-parameter configuration for training.

Hyper-parameter	Value
Actor's learning rate (λ_Q)	0.0003
Critic's learning rate (λ_π)	0.0003
Alpha learning rate (λ_α)	0.0003
Batch size	1024
Step time	0.2 s
Max step per episode	100
Update iteration	10 per episode

Fig. 9 illustrates the designed policy and Q-value networks. A squashed Gaussian policy is used to infer actions in a continuous space [35]. To model the policy network, we use two sequential fully connected (FC) layers with ReLU nonlinearity, each consisting of 128 neurons, and two branched FC layers with 9 neurons that encode the state s_t into the mean μ_t and log standard deviation $\log \sigma_t$ of the action $a_t \in \mathbb{R}^9$. The policy samples stochastic actions by a Gaussian distribution in the training phase and directly outputs deterministic actions with the mean in the evaluation phase. Since the sampled or deterministic action is not bounded in a finite range, a nonlinear squashing function, hyperbolic tangent, is used to ensure the action is in $[-1, 1]$ as follows:

$$a_t = \tanh(\mu_t + \sigma_t \odot \epsilon), \quad \epsilon \sim N(0, I), \quad (13)$$

where $\odot$ is the element-wise product. Unlike the policy network, the Q-value network takes both the state s_t and action a_t as input data to evaluate the policy. We design the network with a single FC layer consisting of 128 neurons and ReLU activation to encode state input. The encoded feature is then concatenated with the action and processed by two additional hidden layers to compute the Q-value of the given state and action ($Q(s_t, a_t)$).

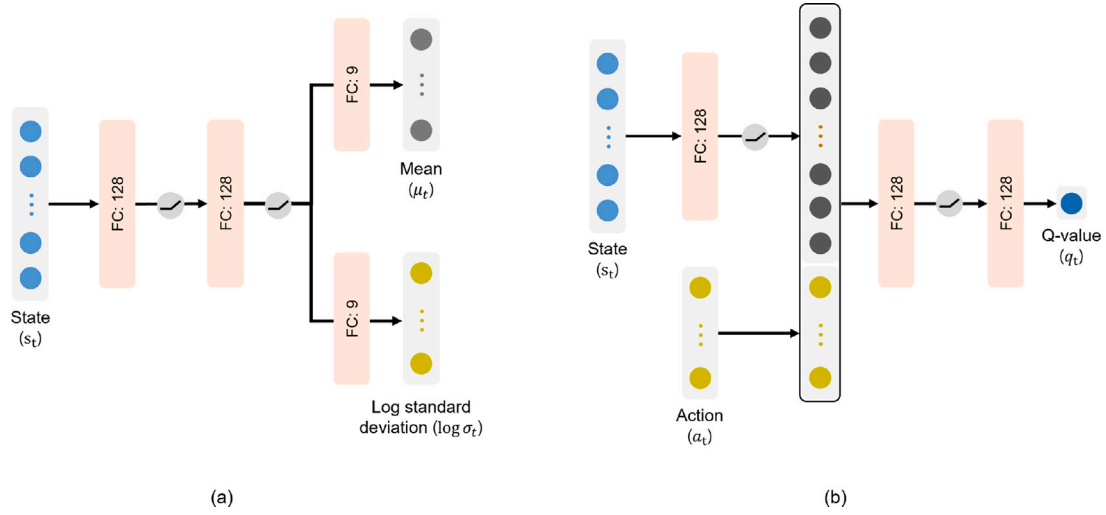


Fig. 9. Actor-critic network architecture: (a) Policy network and (b) Q-value network.

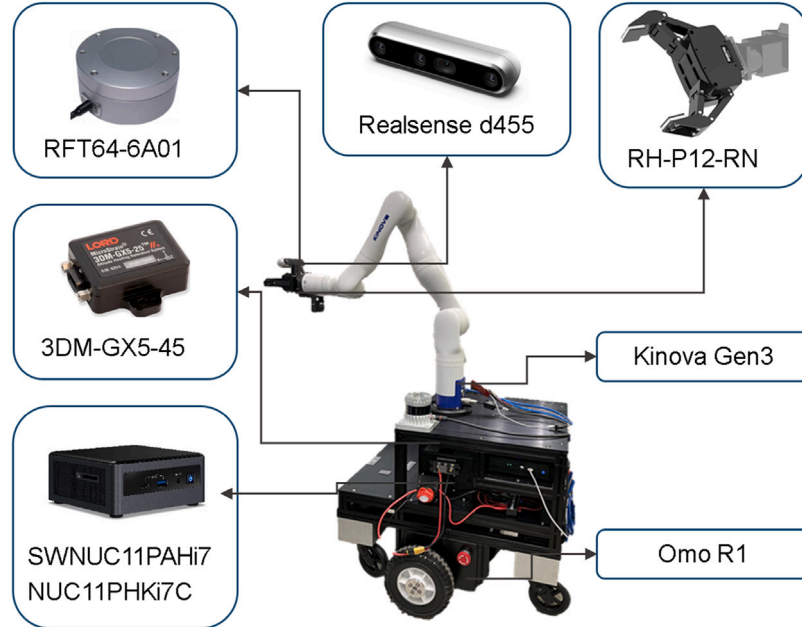


Fig. 10. Hardware configuration and sensors used for the mobile manipulator.

3. Experiment setup

3.1. Mobile manipulator configuration

The hardware configuration of our mobile manipulator system is depicted in Fig. 10. The mobile manipulator consists of the Kinova Gen3 manipulator with 7 degrees of freedom and the Omo R1 mobile base platform equipped with differential wheels. Robotis two-finger gripper (PH-P12-RN) is used as the robot's hand, which is capable of easily grasping various types of door handles including levers, door knobs, and bar handles. The gripper has a payload capacity of 5 kg, a gripping force of 170N, and weighs only 500 g, making it suitable for the mobile manipulator due to its light weight compared to other finger designs. A force/torque sensor (RFT64-6A01) for external force measurement and an Intel RealSense D455 depth camera for object recognition is attached between the manipulator and the gripper. The robot utilizes two Intel NUCs as onboard computers, with one of them, the NUC11PHKi7C, running the deep learning algorithms.

The robot system was implemented and evaluated on ROS1 (Melodic). Open-source drivers from `ros_{kortex}` [37] and `omo_{r1}` [38] were employed for both the robot's manipulator and mobile base to transfer joint and wheel commands from ROS nodes to the robot.

3.2. Training environment

We configured the training environment using the GAZEBO simulator with ROS for communication between the robot and the simulated environment. The state was then preprocessed and fed to the policy model, which inferred and transferred actions through ROS messages. All computed data of the state, action, and reward pairs were collected in a replay buffer for training. We randomized the width of doors (0.75 to 1.5 m) and the initial orientation of the base platform (-0.3 to 0.3 radians) in the environment at the beginning of each episode, to enhance the robustness of our policy model as in Fig. 11. We also added observation noise to the LiDAR data as Gaussian distribution with 0.04 standard deviation for a more realistic configuration.

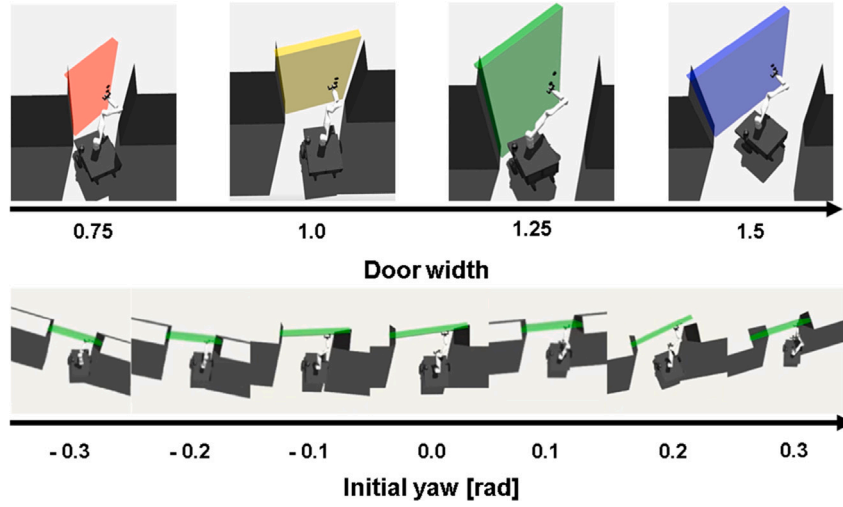


Fig. 11. Training scenarios including four different door widths and seven initial orientations.

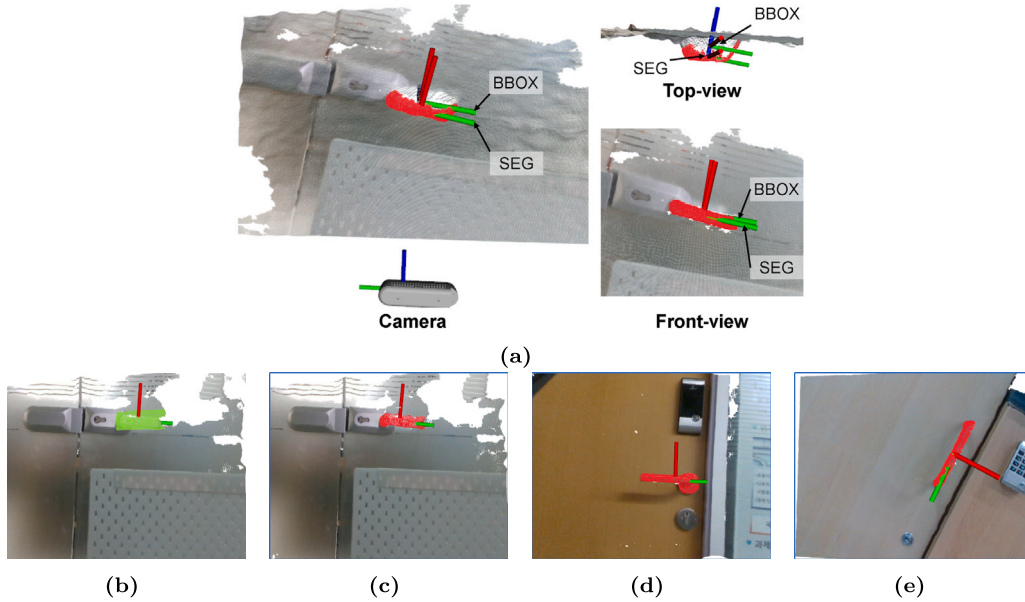


Fig. 12. Detected handle point cloud and its pose estimation. (a) Pose estimation using the bounding box-based (BBOX) and semantic segmentation-based (SEG) algorithms. (b) Detected handle point cloud (green) and estimated handle pose using the bounding box. (c) Detected handle point cloud (red) and estimated handle pose using our semantic segmentation approach. (d)–(e) Handle detection and pose estimation for different handle shapes and types with the semantic segmentation-based algorithm. The estimated poses shown in the figure are in the camera frame. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

4. Results

4.1. Handle pose estimation

The handle pose estimation algorithm was tested on handles with different shapes. Fig. 12(a) and (c–e) demonstrate the successful extraction of the target point cloud (highlighted in red) from the depth image. As the image segmentation which is used as the mask to filter the target handle, was able to detect objects regardless of their shape and size, the detection algorithm was also able to detect various handles. Moreover, the algorithm showed its effectiveness in estimating the handle pose across different camera orientations. It successfully estimated the pose even when the camera pose was tilted, in contrast to the algorithm that relied on using a bounding box as a mask Fig. 12(b). Compared

to the bounding box method, which could not accurately separate the background from the target, the segmentation-based approach exhibited higher accuracy in depth (0.027 m) and orientation estimation. The algorithm also successfully detected the handle on both transparent and non-transparent doors.

4.2. Training in virtual environment

Training the agent for door opening with RL in simulation went through 1000 episodes, with a maximum of 100 steps per episode. Episodes were terminated either when the robot opened the door up to a maximum angle of $\pi/2$ or when a collision occurred. Fig. 13 presents the average training reward (a) and the final door angle achieved

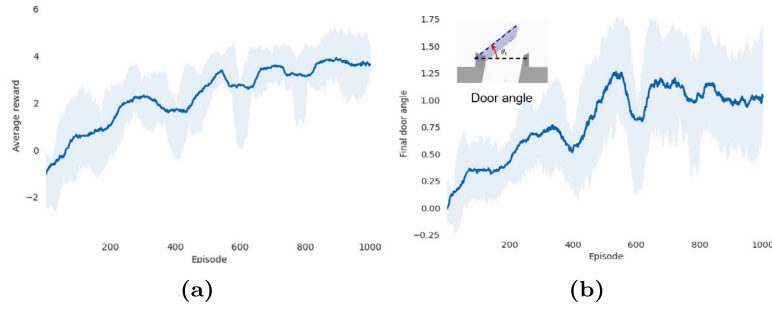


Fig. 13. Training results. (a) average reward, and (b) the final door angle of each episode.

Table 2

Quantitative evaluation of door opening in simulated scenarios. Door widths: Wide (1.25 m), Mid (1.0 m), Narrow (0.75 m). SRate and T represent the success rate (%) and the time (s) taken to complete the door opening process accordingly.

Initial yaw	Door width							
	Wide		Mid		Narrow		0.9 m (unseen)	
	SRate ↑	T ↓	SRate ↑	T ↓	SRate ↑	T ↓	SRate ↑	T ↓
Left	1.0	8.45	1.0	8.18	1.0	11.18	1.0	7.80
Center	1.0	11.66	1.0	7.66	1.0	6.28	1.0	10.54
Right	1.0	9.08	1.0	5.38	0.50	14.62	0.77	13.36
Average	1.0	9.73	1.0	7.07	0.83	10.69	0.92	10.57

in each episode (b). The robot was trained only for the CCW push-open door as the model could not converge when given scenarios with diverse door opening directions.

To evaluate the proposed learning-based policy, we conducted tests on twelve scenarios, nine of which were training scenarios, and three were unseen scenarios. The scenarios included four types of door widths (0.75, 0.9, 1.0, and 1.25 m), with one unseen width of 0.9 m. The initial orientation of the robot relative to the target door was chosen from among three different yaw angles: Left (-0.3), Center (0.0), and Right (0.3). The initial position of the robot also varied up to 0.2 m along the x-axis and 0.05 m along the y-axis. During the evaluation, an episode was considered successful if the robot opened the door wider than its width of 0.66 m. Conversely, an episode was considered a failure if the robot reached the time limit or experienced a collision.

The evaluation results in Table 2 show that the robot successfully opened the door in most cases. The robot did not achieve a 1.0 success rate only for the narrowest door (0.75 m) and the door with unseen width (0.9 m) that was tilted to the right side. In particular, it showed the lowest success rate of 0.5 with the narrow door tilted to the right, as the robot kept having collisions with the door and the wall due to the initial pose of its elbow joint being close to the right side of the wall. However, the robot achieved a 1.0 success rate for non-tilted and left-tilted 0.9 m width doors, which shows that the robot is able to open doors that were not part of its training data.

The average opening time was approximately 10 s for wide, narrow, and 0.9 m doors, while the time for the 1.0 m door was 7.07 s. The tilted-narrow door took the longest time to open, with an average of 14.62 s, while the non-tilted narrow door was the quickest at 6.27 s. This difference was due to the initial pose of the manipulator joints, which caused the robot to move its joints further from the wall to avoid collisions during the door opening motion. The maximum time spent opening the door was 14.8 s, which occurred with the tilted-narrow and 0.9 m doors, and the minimum time spent was 4.8 s for the non-tilted narrow door.

4.3. Door opening in the real-world

The system's performance was evaluated based on the stability of joint velocities, force/torque applied at the end-effector, and efforts on

the manipulator's joints. Fig. 14 illustrates the real-world test results for both the adaptive position-force-based and RL-based door opening motions. The test was conducted on a Push-CCW type door with a width of 0.9 m and a lever type (type1) handle, where $\psi^x = 1$ and $\psi^y = 1$. Since the RL algorithm was only trained for the door opening and passing motion, the other phases (ST1-ST3) were executed in the same way as the adaptive position-force-based system.

The average force and torque applied at the end-effector were 8.82 N and 1.29 N m for the adaptive position-force control and 5.75 N and 0.72 N m for the RL-based control. When using the adaptive position-force control, the robot did not generate accurate trajectories due to lack of information, but rather moved roughly towards the opening direction of the door and adjusted to the external forces by feedback control. This resulted in higher external forces at the end-effector, as the robot did not estimate the force it would cause by moving in such constrained environment. However, the RL agent was able to reduce the maximum force by 3.27 times and the maximum torque by 1.29 times, which indicates that the robot was able to learn actions that could reduce the external force and torque. The average effort on the joints was 6.73 N m and 5.41 N m for the adaptive position-force-based system and the RL-based system, respectively, which also shows that the RL agent effectively reduced torques applied on each joint.

The average joint acceleration of the adaptive position-force-based system was 0.042 sec^{-2} , while it was 0.023 sec^{-2} for the RL-based system, which was almost half that of the adaptive position-force-based system. The gap was pronounced at the end-effector joint ($\hat{q}_7$), where the average angular acceleration was 0.073 sec^{-2} for the adaptive position-force-based method and 0.04 sec^{-2} for the RL-based method. The method based on adaptive position-force control exhibited high joint velocity, reaching up to 0.44 rad/s on average, when the mobile base was moving (as highlighted in Fig. 14). This occurred because the manipulator reacted to compensate for the external force generated by the movement of the mobile base. The end-effector force-torque graph for the passive axes, measured at the same time, shows low values. This indicates that, despite the movement of the mobile base, the manipulator was able to generate compliant motion. Furthermore, it showed higher joint acceleration before the robot identified the horizontal opening direction (ψ^y) of the door, as the robot reacted more

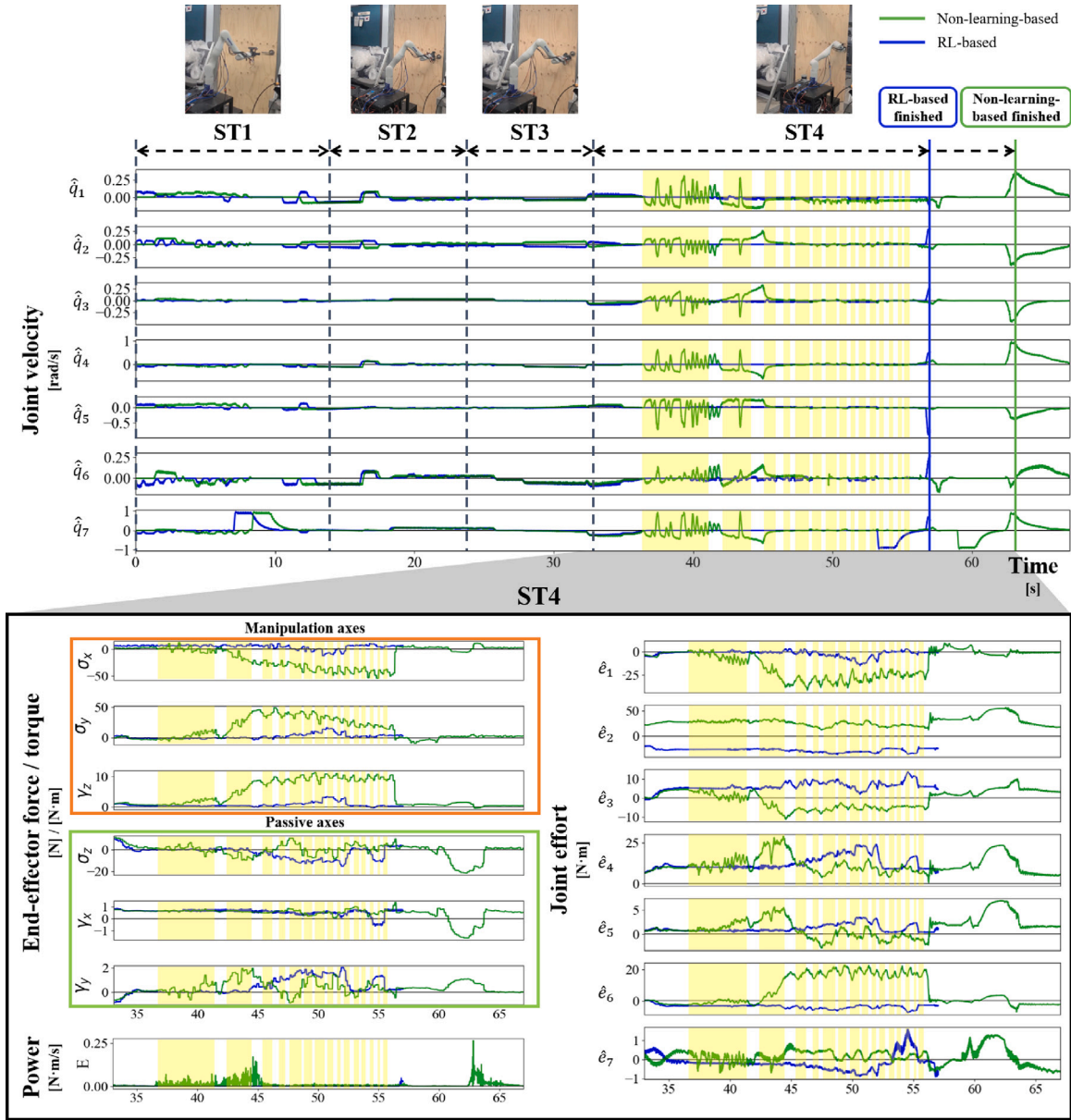


Fig. 14. Real-world demonstration and performance comparison of adaptive position-force-based and RL-based door opening motions. The green lines depict the outcomes of the adaptive position-force control-based system, while the blue lines illustrate the results of the RL-based system. The graphs within the orange box represent the force-torque values of manipulation axes, where force needs to be exerted. Meanwhile, the graphs within the green box display the values of passive axes, involving only compliant movement. The yellow areas indicate instances when the mobile base moved in the adaptive control-based system. In contrast, the mobile base in the RL-based system constantly exhibited movement. Images of the robot in each subtask phase are presented at the top of the figure. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

sensitively to external forces. However, the robot became more stable afterward, indicating that the robot was able to open the door smoothly once it identified the proper opening direction.

The total power consumption during ST4 was 11.11 N m for the RL-based system and 160.95 N m for the non-learning-based system. The RL-based system, during the learning process, successfully reduced unnecessary actions through action scale penalties, resulting in more stable and smaller joint speed values. In contrast, as the non-learning-based system is designed to respond sensitively to external forces, it generated higher joint speed values when external force was exerted. The power consumption graphs for each method during the execution of subtask 4 are presented in the bottom-left corner of Fig. 14. Additionally, the RL-based system completed ST4 19% more quickly than the adaptive-based method.

4.4. Versatile door opening with adaptive position-force control

The adaptive position-force control-based system was evaluated with doors of various kinematics, such as different widths, opening directions ψ^x , ψ^y , and handle types, as illustrated in Figs. 15–17. In Fig. 15, the robot opened a Push-CW type door ($\psi^x = 1$, $\psi^y = -1$) with a width of 0.9 m. In ST1, the robot approached the door from 2.5 m away while simultaneously estimating the 6D pose of the handle and aligning with it. After aligning with the handle, the robot reached and unlatched the handle based on its type (type1) using the end-effector in ST2. Our method properly unlatched the handle until the force at the end-effector detected the unlock based on the real-time force/torque data. In ST3, the robot estimated the kinematics of the door, including its opening direction using force measurement. Finally, the robot opened the door based on the estimated door kinematics (ST4) until the door was fully opened, allowing the robot to pass through during the terminal stage.

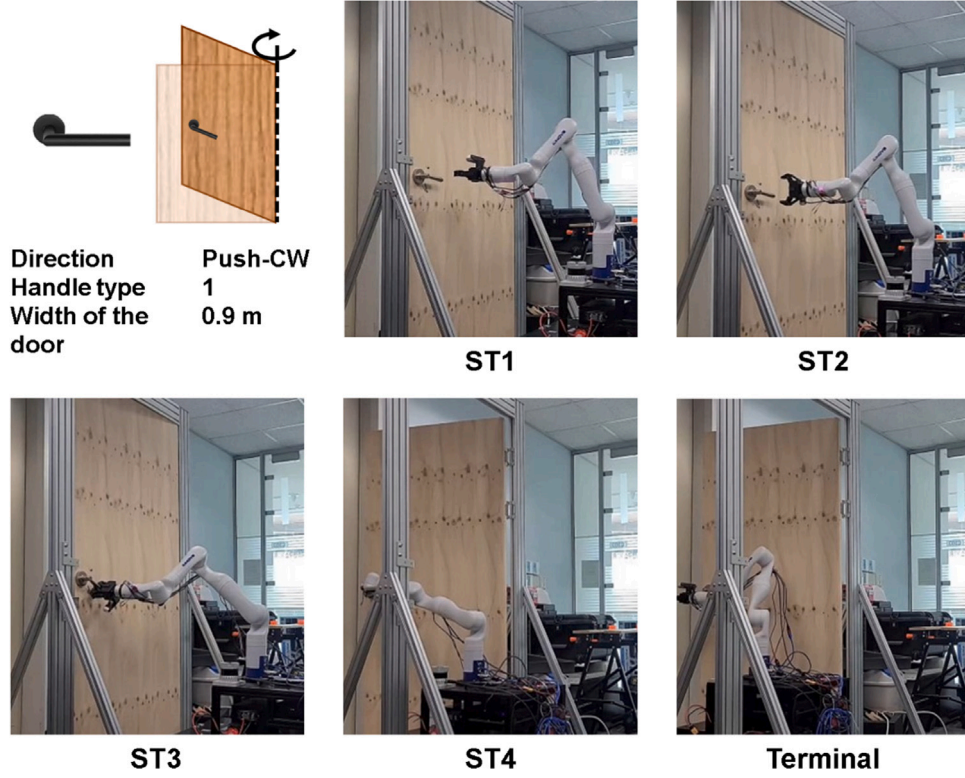


Fig. 15. The opening process for a Push-CW type door with a type1 handle starts with the robot detecting and classifying the handle in ST1, and preparing to unlatch it in ST2. Once the handle is unlatched, the robot determines the door's opening direction in ST3. Since the door is a Push-CW type, the robot moves its end-effector in the front-left direction while compensating for external forces generated during the opening motion using adaptive position-force control in ST4. When the door is fully open, the robot returns to its manipulator pose and initiates autonomous driving mode as the terminal state.

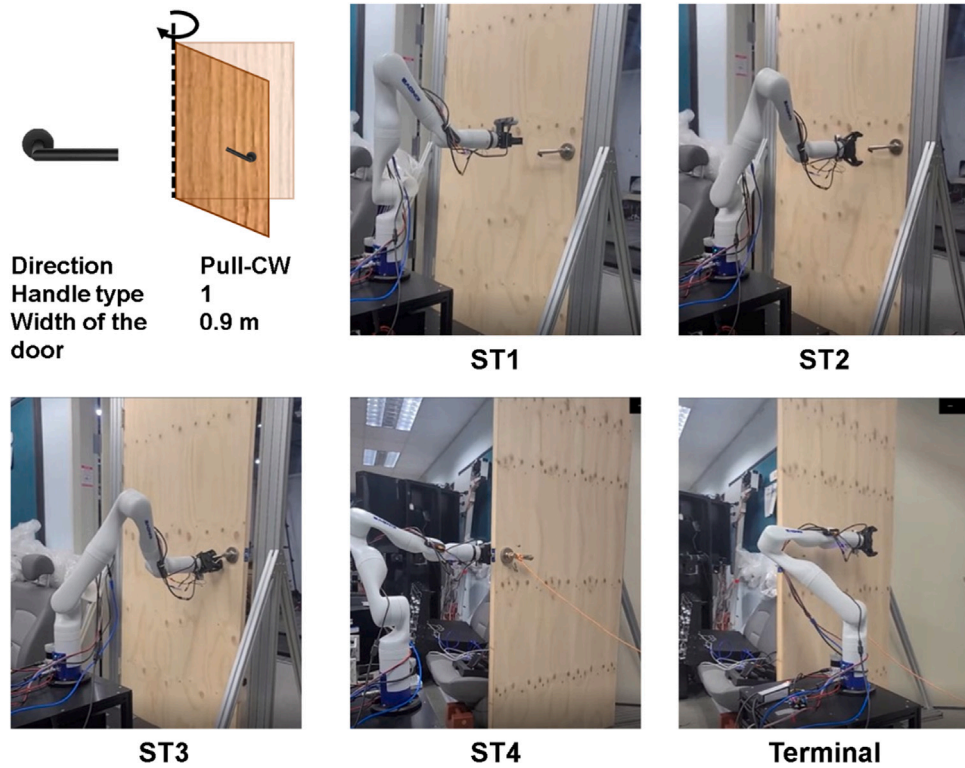


Fig. 16. Opening Pull-CW type door with type1 handle. The robot estimates the pose of the handle at ST1 and grabs and unlatches the handle according to its type in ST2. The robot identifies the door is a Pull-CW type in ST3. While opening the door, the mobile base moves backward to avoid collision with the door and generate the manipulator workspace in ST4. When the door is fully opened, the mobile robot moves forward to block the door from closing and release the end-effector and initialize the manipulator pose.

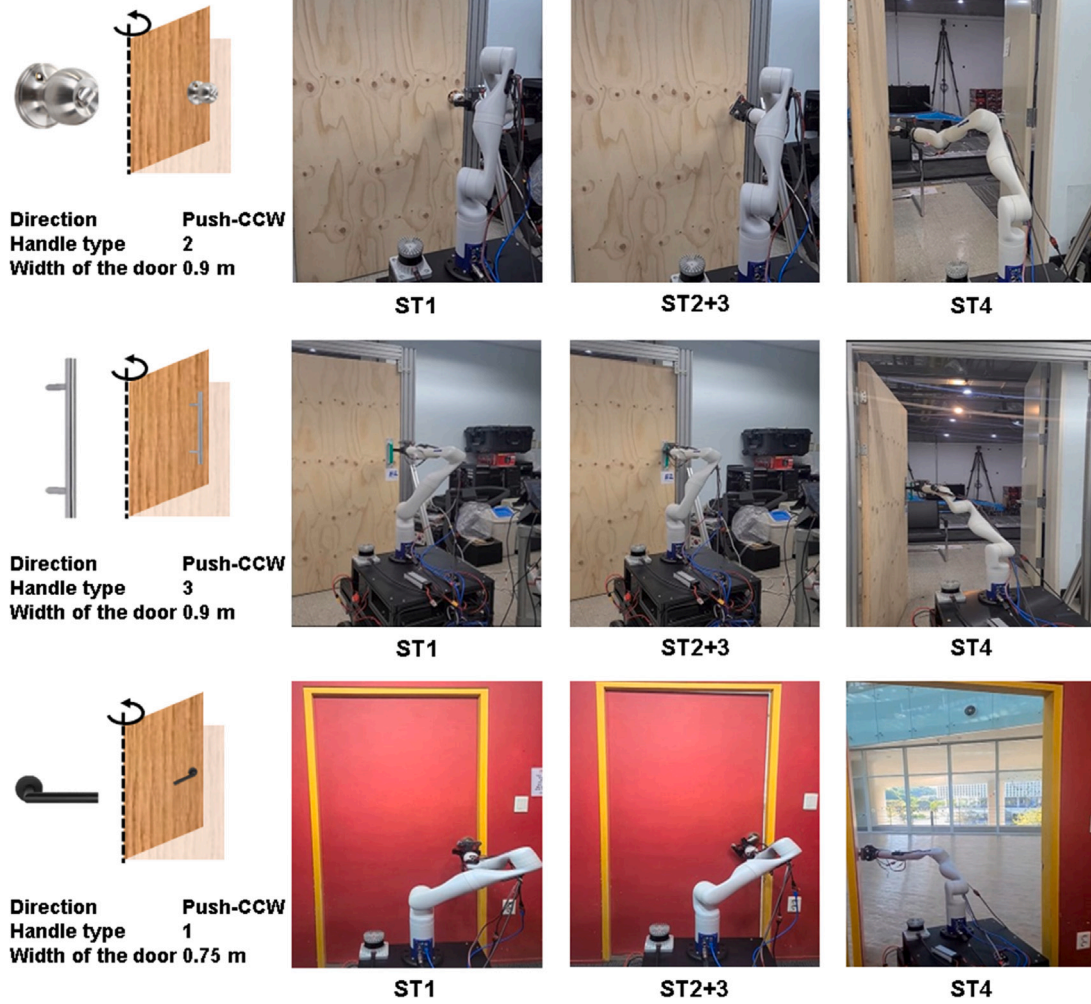


Fig. 17. Opening Push-CW type door with type1, type2, and type3 handle. If the target handle is type2, the robot grasps the handle and turns it until σ_{ee} reaches Ω_{ee} and A_{ee} . However, when the handle is type3, the robot grasps the handle and directly moves on to ST3 without unlatching motion. The robot successfully passes through the door with different widths and different shapes of handles (type1) in the last row of the figure using adaptive motion.

In Fig. 16, our robot succeeded in opening a more challenging scenario of the Pull-CW type door ($\psi^x = -1$, $\psi^y = 1$). Similar to the scenario of the Push-CW type door, the robot drew close to the door and unlocked the handle (type1) during ST1–ST2 while estimating the door kinematics in the process of ST3. Since our method determined it as a pulling door, the end-effector of the manipulator performed pulling actuation while the base robot reactively maneuvered backward based on the real-time workspace checking algorithm. After opening the door so that our mobile base robot had enough drivable region, the robot drove forward to go through the door, and the end-effector returned to its initial pose.

The door opening system was also tested on type2 and type3 handles in Fig. 17. The procedure was the same as the door with type1 handle for both types, except for ST2 in the type3 handle as it does not need to be unlocked. To test the adaptiveness of the door opening motion, doors with different widths were tested in the bottom figures of Fig. 17. The robot successfully opened the type1 handle with different shapes and sizes and was able to adaptively open the narrow door with adaptive position-force control.

5. Conclusion

In this paper, we present a mobile manipulator system capable of full sequence opening and passing doors without prior knowledge or human guidance. The task was decomposed into four subtasks,

and relevant information such as the pose, type of the handle, and the opening direction of the door was identified online during each procedure. Also, we propose two most popular approaches that enable the mobile manipulator to open doors: an adaptive position-force-based control method and a reinforcement learning-based method.

The adaptive position-force controller enabled the robot to open doors in four different directions and widths, demonstrating its flexibility in terms of different door trajectories without prior knowledge. This adaptability also makes it suitable for opening more diverse doors. On the other hand, the RL-based control system considered not only forces but also the position, velocity, and surrounding obstacles, enabling it to perform well-defined tasks. It achieved stable behavior and high energy efficiency while opening doors with varying widths, initial poses, and previously unseen kinematics. However, as the task scope expands, it has been observed that RL failed to train the policy, revealing limitations in terms of scalability to diverse tasks.

The future work of this research will focus on dynamically addressing doors with inconsistent opening directions, such as those requiring alternating pushing and pulling actions. Additionally, we aim to address challenges posed by moving doors, including revolving ones. Further studies may expand beyond door-opening tasks to explore connections with other activities, such as using elevators and loading packages. Specifically, integrating driving mode with door opening, considering the relative pose of the target door, and enhancing the search for the target door could make this study more practical. This broader scope is intended to enhance the robot's versatility for general indoor use.

CRediT authorship contribution statement

Gyuree Kang: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Hyunki Seong:** Formal analysis, Methodology, Software, Visualization, Writing – original draft, Writing – review & editing. **Daegy Lee:** Methodology, Software, Writing – original draft. **David Hyunchul Shim:** Conceptualization, Funding acquisition, Investigation, Project administration, Resources, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: D. Hyunchul Shim reports financial support was provided by Agency For Defense Development.

Data availability

No data was used for the research described in the article.

Acknowledgments

This research was supported by the Challengeable Future Defense Technology Research and Development Program through the Agency For Defense Development (ADD) funded by the Defense Acquisition Program Administration (DAPA) in 2023 (No. 915027201).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.robot.2024.104760>.

References

- [1] D. Lee, G. Kang, B. Kim, D. Shim, Assistive delivery robot application for real-world postal services, *IEEE Access* 9 (2021) 141981–141998.
- [2] S. Srinivas, S. Ramachandiran, S. Rajendran, Autonomous robot-driven deliveries: A review of recent developments and future directions, *Transp. Res. E: Logist. Transp. Rev.* 165 (2022) 102834.
- [3] H. Tanner, K. Kyriakopoulos, N. Krikellis, Advanced agricultural robots: kinematics and dynamics of multiple mobile manipulators handling non-rigid material, *Comput. Electron. Agricult.* 31 (2001) 91–105.
- [4] A. Iriondo, E. Lazkano, L. Susperregi, J. Urain, A. Fernandez, J. Molina, Pick and place operations in logistics using a mobile manipulator controlled with deep reinforcement learning, *Appl. Sci.* 9 (2019) 348.
- [5] M. Yang, E. Yang, R. Zante, M. Post, X. Liu, Collaborative mobile industrial manipulator: a review of system architecture and applications, in: 2019 25th International Conference On Automation And Computing, ICAC, 2019, pp. 1–6.
- [6] L. Peterson, D. Austin, D. Kragic, High-level control of a mobile manipulator for door opening, in: Proceedings. 2000 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2000)(Cat. No. 00CH37113), Vol. 3, 2000, pp. 2333–2338.
- [7] S. Ahmad, H. Zhang, G. Liu, Multiple working mode control of door opening with a mobile modular and reconfigurable robot, *IEEE/ASME Trans. Mechatron.* 18 (2012) 833–844.
- [8] C. Ma, H. Gao, L. Ding, J. Tao, K. Xia, H. Yu, Z. Deng, Optimal energy consumption for mobile manipulators executing door opening task, in: Mathematical Problems In Engineering, 2018, (2018).
- [9] M. Stuede, K. Nuelle, S. Tappe, T. Ortmaier, Door opening and traversal with an industrial cartesian impedance controlled mobile robot, in: 2019 International Conference on Robotics and Automation, ICRA, 2019, pp. 966–972.
- [10] M. Arduengo, C. Torras, L. Sents, Robust and adaptive door operation with a mobile robot, *Intell. Serv. Robot.* 14 (2021) 409–425.
- [11] M. Minniti, F. Farshidian, R. Grandia, M. Hutter, Whole-body mpc for a dynamically stable mobile manipulator, *IEEE Robot. Autom. Lett.* 4 (2019) 3687–3694.
- [12] J. Luo, E. Solowjow, C. Wen, J. Ojea, A. Agogino, A. Tamar, P. Abbeel, Reinforcement learning on variable impedance controller for high-precision robotic assembly, in: 2019 International Conference on Robotics and Automation, ICRA, 2019, pp. 3080–3087.
- [13] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. Ojea, E. Solowjow, S. Levine, Residual reinforcement learning for robot control, in: 2019 International Conference on Robotics and Automation, ICRA, 2019, pp. 6023–6029.
- [14] C. Beltran-Hernandez, D. Petit, I. Ramirez-Alpizar, K. Harada, Variable compliance control for robotic peg-in-hole assembly: A deep-reinforcement-learning approach, *Appl. Sci.* 10 (2020) 6923.
- [15] C. Wang, Q. Zhang, Q. Tian, S. Li, X. Wang, D. Lane, Y. Petillot, S. Wang, Learning mobile manipulation through deep reinforcement learning, *Sensors* 20 (2020) 939.
- [16] J. Kindle, F. Furrer, T. Novkovic, J. Chung, R. Siegwart, J. Nieto, Whole-body control of a mobile manipulator using end-to-end reinforcement learning, 2020, arXiv Preprint arXiv:2003.02637.
- [17] D. Honerkamp, T. Welschehold, A. Valada, Learning kinematic feasibility for mobile manipulation through deep reinforcement learning, *IEEE Robot. Autom. Lett.* 6 (2021) 6289–6296.
- [18] C. Sun, J. Orbi, C. Devin, B. Yang, A. Gupta, G. Berseth, S. Levine, Fully autonomous real-world reinforcement learning with applications to mobile manipulation, in: Conference On Robot Learning, 2022, pp. 308–319.
- [19] R. Liu, F. Nageotte, P. Zanne, M. Mathelin, B. Dresp-Langley, Deep reinforcement learning for the control of robotic manipulation: a focussed mini-review, *Robotics* 10 (2021) 22.
- [20] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, S. Levine, How to train your robot with deep reinforcement learning: lessons we have learned, *Int. J. Robot. Res.* 40 (2021) 698–721.
- [21] S. Gu, E. Holly, T. Lillicrap, S. Levine, Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates, in: 2017 IEEE International Conference on Robotics and Automation, ICRA, 2017, pp. 3389–3396.
- [22] B. Nemec, L. Žlajpah, A. Ude, Door opening by joining reinforcement learning and intelligent control, in: 2017 18th International Conference on Advanced Robotics, ICAR, 2017, pp. 222–228.
- [23] Y. Urakami, A. Hodgkinson, C. Carlin, R. Leu, L. Rigazio, P. Abbeel, Doorgym: A scalable door opening environment and baseline agent, 2019, arXiv Preprint arXiv:1908.01887.
- [24] Y. Wang, L. Wang, Y. Zhao, Research on door opening operation of mobile robotic arm based on reinforcement learning, *Appl. Sci.* 12 (2022) 5204.
- [25] H. Ito, K. Yamamoto, H. Mori, T. Ogata, Efficient multitask learning with an embodied predictive model for door opening and entry with whole-body control, *Sci. Robot.* 7 (2022) eaax8177.
- [26] X. Lai, J. Liu, L. Jiang, L. Wang, H. Zhao, S. Liu, X. Qi, J. Jia, Stratified transformer for 3d point cloud segmentation, in: Proceedings of The IEEE/CVF Conference On Computer Vision And Pattern Recognition, 2022, pp. 8500–8509.
- [27] H. Liu, R. Soto, F. Xiao, Y. Lee, Yolactedge: Real-time instance segmentation on the edge, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, 2021, pp. 9579–9585.
- [28] M. Fischler, R. Bolles, Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography, *Commun. ACM* 24 (1981) 381–395.
- [29] K. Lynch, F. Park, Modern Robotics, Cambridge University Press, 2017.
- [30] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: International Conference On Machine Learning, 2018, pp. 1861–1870.
- [31] F. Fuchs, Y. Song, E. Kaufmann, D. Scaramuzza, P. Duerr, Super-human performance in gran turismo sport using deep reinforcement learning, 2020, arXiv Preprint arXiv:2008.07971.
- [32] H. Seong, C. Jung, S. Lee, D. Shim, Learning to drive at unsignalized intersections using attention-based deep reinforcement learning, in: 2021 IEEE International Intelligent Transportation Systems Conference, ITSC, 2021, pp. 559–566.
- [33] C. Yang, J. Yang, X. Wang, B. Liang, Control of space flexible manipulator using soft actor-critic and random network distillation, in: 2019 IEEE International Conference on Robotics and Biomimetics, ROBIO, 2019, pp. 3019–3024.
- [34] E. Prianto, M. Kim, J. Park, J. Bae, J. Kim, Path planning for multi-arm manipulators using deep reinforcement learning: Soft actor-critic with hindsight experience replay, *Sensors* 20 (2020) 5911.
- [35] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, et al., Soft actor-critic algorithms and applications, 2018, arXiv Preprint arXiv:1812.05905.
- [36] S. Fujimoto, H. Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: International Conference On Machine Learning, 2018, pp. 1587–1596.
- [37] Kinovarobotics ros_kortex. GitHub Repository, 2018, https://github.com/Kinovarobotics/ros_kortex.git.
- [38] omorobot omo_r1. GitHub Repository, 2021, https://github.com/omorobot/omo_r1.git.



Gyuree Kang received her B.S. degree in Mechanical Engineering from Sungkyunkwan University, Suwon, Gyeonggi, South Korea, in 2020, and her M.S. degree in the Robotics Program from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2022. She is currently pursuing a Ph.D. degree in Electrical Engineering at KAIST. Her research interests include autonomous systems, robotics, and task and motion planning.



Hyunki Seong received his B.S. degree in Mechanical Engineering from Inha University, Incheon, South Korea, in 2019, and his M.S. degree in Division of Robotics from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2021. He is currently pursuing a Ph.D. degree in Electrical Engineering at KAIST. His research interests include reinforcement learning and imitation learning for autonomous driving and robotics.



Daegyu Lee received his B.S. degree in Automotive Engineering from Kookmin University, Seoul, South Korea, in 2018, and received M.S. and Ph.D. degrees in the Division of Future Vehicle and Electrical Engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2020 and 2024, respectively. He is currently a researcher at the Electronics and Telecommunications Research Institute (ETRI), Daejeon, South Korea. His research interests include autonomous systems and localization, focusing on robust and learning-based approaches.



D. Hyunchul Shim received his B.S. and M.S. degrees in Mechanical Design and Production Engineering from Seoul National University, Seoul, South Korea, in 1991 and 1993, respectively, and his Ph.D. degree in Mechanical Engineering from the University of California at Berkeley, Berkeley, CA, USA, in 2000. He worked with the Hyundai Motor Company and Maxtor Corporation from 1993 to 1994 and from 2001 to 2005, respectively. In 2007, he joined the Department of Aerospace Engineering, KAIST, Daejeon, South Korea, and is currently a tenured Professor with the Department of Electrical Engineering, and Adjunct Professor, Graduate School of AI, KAIST. His research interests include control systems, autonomous vehicles, and robotics. He is also the Director of the Korea Civil RPAS Research Center.